

Meetspace.Nirmaan — Office Meeting Room Management Portal

Portal: `meetspace.nirmaan.org`

1. PROJECT OBJECTIVE

Build an enterprise-grade, production-ready full-stack **Office Meeting Room Management System** exclusively for **Nirmaan Organization**.

The system must manage:

- Meeting Rooms
- Room Availability
- Meeting Bookings
- Booking Approvals
- Departments
- Users
- User Roles
- Granular Permissions
- Notifications
- Email Notifications
- Meeting Calendar
- Room Maintenance
- Room Utilization
- Reports
- Activity Logs
- Administrative Settings

This is an **internal Nirmaan enterprise application**.

STRICT SCOPE

Do NOT add:

- Website Analytics
- Website Master
- SEO Analytics
- Visitors
- Page Views
- Sessions
- Bounce Rate
- Website Performance Analytics
- Marketing Analytics

- Unrelated CRM modules
- Unrelated HR modules
- Unrelated financial modules

The entire application must focus exclusively on:

Nirmaan Office Meeting Room Management

2. TECHNOLOGY STACK

Frontend

- React 18
- Vite
- Material UI
- MUI X DataGrid
- React Router
- Axios
- ApexCharts
- FullCalendar or equivalent enterprise calendar component
- React Hook Form
- Yup/Zod validation

Backend

- Node.js
- Express.js
- Sequelize ORM
- MySQL
- JWT Authentication
- bcrypt
- Nodemailer / enterprise email service
- ExcelJS
- PDFKit

Architecture

Use a modular architecture so the complete **Meeting Room module can later be separated and moved to another Nirmaan portal without breaking the application.**

3. AUTHENTICATION

Implement secure authentication.

Support:

- Login
- Logout
- JWT access token
- Refresh token where appropriate
- Password hashing using bcrypt
- Password reset
- Session management
- Account active/inactive status
- Failed login tracking
- Last login
- Login activity logging

Every protected API must validate:

1. Authentication
2. User status
3. Role
4. Permission
5. Resource-level access where configured

Never rely only on frontend permission hiding.

Backend APIs must enforce permissions.

4. USER ROLES

Default system roles:

Super Admin

Complete unrestricted access.

Can:

- Manage all users
- Manage all roles
- Create custom roles
- Manage all permissions
- Manage departments
- Manage rooms
- Manage facilities
- Manage maintenance
- Manage bookings

- Manage approvals
- Configure approval workflows
- Configure email notifications
- Configure system settings
- View all reports
- View all activity logs
- Configure organization-wide access

Admin

Administrative access according to permissions assigned by Super Admin.

Admin can be given:

- User Management
- Department Management
- Room Management
- Booking Management
- Approval Management
- Reports
- Notifications
- Maintenance
- Activity Logs

However, each permission must be independently configurable.

Department Head / Manager

Normally can:

- View department bookings
- Approve department bookings
- View department users
- View department reports

Access must be configurable.

Employee / User

Normally can:

- View available rooms
- Create booking requests
- View own bookings
- Modify own bookings where allowed
- Cancel own bookings where allowed
- Receive notifications
- View calendar

Viewer

Read-only access only.

Cannot:

- Create
- Edit
- Delete
- Approve
- Cancel
- Manage users
- Manage rooms

5. GRANULAR PERMISSION SYSTEM

This is a critical part of the application.

Do NOT use only simple roles such as Admin/User.

Implement:

Role → Module → Permission → Scope

Permission Types

Every module can have:

- View
- Create
- Edit
- Delete
- Approve
- Reject
- Cancel
- Export
- Manage
- Configure

Example:

Meeting Rooms

A user can have:

- View Rooms = ON
- Create Room = OFF
- Edit Room = ON
- Delete Room = OFF
- Room Maintenance = ON
- Export Room Data = OFF

Therefore the user can manage existing rooms but cannot create/delete rooms.

6. MODULE-LEVEL PERMISSIONS

Create a Permission Management screen.

Modules:

- Dashboard
- Calendar
- Book Meeting
- My Bookings
- Approval Requests
- Meeting Rooms
- Room Facilities
- Room Maintenance
- Departments
- Users
- Roles & Permissions
- Reports
- Notifications
- Activity Logs
- System Settings

Each module must have configurable permissions.

Example UI:

Module	View	Create	Edit	Delete	Approve	Export	Manage
Dashboard	ON	-	-	-	-	ON	ON
Meeting Rooms	ON	ON	ON	OFF	-	ON	ON
Users	ON	ON	ON	OFF	-	OFF	ON

Module	View	Create	Edit	Delete	Approve	Export	Manage
Bookings	ON	ON	ON	OFF	ON	ON	ON
Departments	ON	ON	ON	OFF	-	ON	ON
Reports	ON	-	-	-	-	ON	ON

The administrator must be able to turn each permission ON/OFF.

7. RESOURCE-LEVEL / LIMITED ACCESS

Permission management must go beyond module-level permissions.

Support:

Which rooms / departments / buildings can this user manage?

Example:

A user may have:

Meeting Room Management

- View = ON
- Create = OFF
- Edit = ON
- Delete = OFF

Scope

- Building: Main Office
- Floor: 2nd Floor
- Rooms: Conference Room A, Conference Room B

Therefore this user can manage only those rooms.

They cannot edit:

- 1st Floor rooms
 - 3rd Floor rooms
 - Other buildings
-

8. ACCESS SCOPE TYPES

Support:

Organization-wide

User can access everything.

Building-level

User can access rooms within selected buildings.

Floor-level

User can access rooms on selected floors.

Room-level

User can access only selected rooms.

Department-level

User can access only selected departments.

Own-data

User can access only records created by themselves.

Department-data

User can access records belonging to their department.

Custom

Administrator can configure specific resources.

9. EXAMPLE LIMITED ROOM MANAGER

Create a predefined concept:

Room Manager

A Room Manager may be responsible for specific rooms.

Example:

User: Ravi

Permissions:

- Meeting Rooms → View = ON
- Meeting Rooms → Edit = ON
- Meeting Rooms → Maintenance = ON
- Meeting Rooms → Create = OFF
- Meeting Rooms → Delete = OFF
- Bookings → View = ON
- Bookings → Cancel = OFF
- Approvals → Approve = OFF
- Reports → View = ON

Scope:

- Building: Head Office
- Floor: 2
- Rooms:
- Board Room
- Conference Room A
- Conference Room B

Ravi cannot modify any other room.

10. CUSTOM ROLE CREATION

Super Admin must be able to create custom roles.

Example roles:

- Room Manager
- Booking Coordinator
- Facilities Manager
- Department Coordinator
- HR Approver
- Meeting Viewer
- Report Viewer

When creating a role:

Step 1

Enter:

- Role Name
- Role Code
- Description
- Status

Step 2

Select modules.

Step 3

Select permissions.

Step 4

Configure data scope.

Step 5

Assign users.

Step 6

Save role.

11. USER MANAGEMENT — DEEP IMPLEMENTATION

Admin/Super Admin should have a complete User Management module.

User fields:

- Employee Name
- Employee ID
- Email
- Mobile
- Designation
- Department
- Manager
- Department Head
- Role
- Status

- Joining Date
- Profile Image
- Password status
- Last Login
- Account Created Date
- Account Updated Date

User Status

- Active
 - Inactive
 - Suspended
 - Locked
-

12. USER CREATE FLOW

Admin/Super Admin:

Users → Add User

Enter:

- Employee information
- Department
- Designation
- Manager
- Role
- Access permissions
- Scope

Then:

Create User

System should:

1. Validate Employee ID
2. Validate email
3. Check duplicate account
4. Create user
5. Assign role
6. Assign permissions
7. Assign scope
8. Generate activity log
9. Send account notification email where configured

13. USER ACTIVATE / DEACTIVATE

Only authorized Admin/Super Admin users can activate/deactivate users.

Before deactivation:

Show confirmation:

Deactivating this user will prevent login and access to Meetspace.Nirmaan.

When deactivated:

- Login blocked
- Existing permissions disabled
- Historical bookings remain
- Historical activity logs remain
- Existing booking ownership remains for audit purposes
- User should not be deleted automatically

Support reactivation.

14. USER DELETE RULE

Do not allow normal deletion of users who have:

- Bookings
- Approvals
- Activity logs

Instead:

Deactivate user

Hard delete should only be available to Super Admin under controlled conditions.

15. DEPARTMENT MANAGEMENT

Create a complete Department Master.

Fields:

- Department Name
- Department Code
- Department Head
- Deputy / Manager where applicable
- Email
- Status
- Description

Actions:

- Add
- Edit
- View
- Activate
- Deactivate
- Delete where permitted

Department deletion must be blocked if active users/bookings depend on it unless proper reassignment is completed.

16. ROOM MANAGEMENT — DEEP IMPLEMENTATION

Create a complete Meeting Room Master.

Fields:

- Room Name
- Room Code
- Building
- Floor
- Location
- Seating Capacity
- Room Type
- Facilities
- Status
- Image
- Description
- Room Manager
- Department / Owning Department where applicable
- Booking availability
- Maintenance status

Room types:

- Conference Room
- Board Room
- Training Room
- Interview Room
- Meeting Room
- Discussion Room
- Other

17. ROOM FACILITIES

Facilities should be configurable from Admin Settings.

Default facilities:

- Projector
- TV / Display
- Video Conferencing
- Whiteboard
- AC
- Wi-Fi
- Speaker
- Microphone

Super Admin can:

- Add facility
- Edit facility
- Activate/deactivate facility

Do not hard-code facilities throughout the application.

18. ROOM CREATION FLOW

Admin/Super Admin:

Meeting Rooms → Add Room

Step 1:

Basic Information

Step 2:

Location

Step 3:

Capacity

Step 4:

Facilities

Step 5:

Room Manager / Responsible Person

Step 6:

Booking Rules

Step 7:

Image & Description

Step 8:

Review

Step 9:

Create Room

Every room creation must create an activity log.

19. ROOM STATUS

Room statuses:

- Available
- Under Maintenance
- Inactive

Available

Can be booked.

Under Maintenance

Cannot be booked.

Inactive

Hidden from normal availability search.

Historical bookings must remain available.

20. ROOM MAINTENANCE

Create:

Room Maintenance

Fields:

- Room
- Maintenance Type
- Start Date
- Start Time
- End Date
- End Time
- Reason
- Created By
- Status

During maintenance:

The room must automatically disappear from available-room search.

21. BOOKING FLOW

Booking process:

Select Date → Start Time → End Time → Participants → Building → Floor → Required Facilities → Available Rooms → Select Room → Meeting Details → Submit

Fields:

- Meeting Title
- Meeting Date
- Start Time
- End Time
- Duration
- Meeting Room
- Organizer
- Department
- Manager
- Participants
- Participant Names
- Purpose
- Required Facilities
- Notes

22. ROOM AVAILABILITY ENGINE

The availability engine must be the single source of truth.

Before booking:

Check:

1. Room status
2. Maintenance schedule
3. Existing bookings
4. Booking status
5. Time overlap
6. Capacity
7. Required facilities
8. User permissions
9. Building/floor restrictions

Never rely only on frontend availability.

Backend must validate again before creating the booking.

23. DOUBLE BOOKING PREVENTION

Never allow overlapping bookings.

Example:

Existing:

10:00 AM – 11:00 AM

New booking:

10:30 AM – 11:30 AM

Must be rejected.

Use proper backend transaction/locking strategy so simultaneous requests cannot create double bookings.

24. BOOKING APPROVAL WORKFLOW

Configurable workflow.

Default:

Employee → Department Head / Manager → HR/Admin → Approved

But Super Admin must be able to configure:

Workflow A

Employee → Approved

Workflow B

Employee → Department Head → Approved

Workflow C

Employee → Admin → Approved

Workflow D

Employee → Department Head → Admin → Approved

Approval workflow should be configurable by:

- Department
- Meeting type

- Room
 - Booking duration
 - Organization rules
-

25. BOOKING STATUS VS APPROVAL STATUS

These must be separate.

Booking Status

- Draft
- Submitted
- Confirmed
- Cancelled
- Completed

Approval Status

- Not Required
- Pending Department Approval
- Pending Admin Approval
- Approved
- Rejected

Do not mix both statuses into one database field.

26. EMAIL — BOOKING REQUEST

When a user submits a booking requiring approval:

Send email immediately.

To

Approver / Department Head / configured approval group.

CC

Where configured.

Email Content

- Meeting Title

- Organizer
- Department
- Date
- Start Time
- End Time
- Room
- Participants
- Purpose
- Approval required
- Booking reference number

Include:

View Booking

Approve

Reject

where secure workflow permits.

27. EMAIL — BOOKING CONFIRMATION

After final approval:

Send confirmation email to:

- Organizer
- Participants where configured
- Department Head where configured

Email should contain:

- Booking Reference
- Meeting Title
- Date
- Start Time
- End Time
- Room
- Building
- Floor
- Organizer
- Department
- Participants
- Facilities

- Approval status

Subject example:

Meeting Room Booking Confirmed — [Meeting Title] — [Date]

28. EMAIL NOTIFICATIONS

Support emails for:

- Booking request
- Approval required
- Booking approved
- Booking rejected
- Booking cancelled
- Booking rescheduled
- Booking modified
- Meeting reminder
- Room maintenance
- Room unavailable
- User account creation
- User account activation
- User account deactivation

Email templates must be configurable from Admin Settings.

29. NOTIFICATION CENTER

Header notification bell.

Display:

- Unread count
- Notification type
- Title
- Message
- Date/time
- Read/unread
- Related booking

Actions:

- Mark read
- Mark all as read

- Open related record
-

30. MEETING CALENDAR

Dedicated calendar.

Views:

- Day
- Week
- Month

Events show:

Meeting Title | Room | Time | Organizer | Department | Status

Event colors/visual indicators must use only the approved Nirmaan portal color system.

Click event:

Open complete booking details.

31. MY BOOKINGS

Employee sees:

- Upcoming bookings
- Past bookings
- Pending bookings
- Cancelled bookings

Columns:

- Booking ID
- Meeting Title
- Date
- Time
- Room
- Department
- Status
- Approval Status
- Organizer

Actions depend on permissions.

32. APPROVAL REQUESTS

Approvers see only records they are authorized to approve.

Filters:

- Pending
- Approved
- Rejected
- Date
- Department
- Room
- Organizer

Actions:

- View
- Approve
- Reject
- Comment

Every approval action must create:

1. Approval record
2. Activity log
3. Notification
4. Email where configured

33. ADMIN SETTINGS — DEEP MODULE

Create a complete:

Settings

Only users with Settings permission can access it.

Settings sections:

Organization

- Organization Name
- Logo
- Portal Name
- Time Zone
- Date Format
- Time Format

Booking Rules

- Minimum booking duration
- Maximum booking duration
- Advance booking days
- Same-day booking allowed
- Weekend booking allowed
- Holiday booking allowed
- Cancellation deadline
- Rescheduling rules
- Participant limits

Approval Settings

- Approval required ON/OFF
- Department Head approval
- Admin approval
- Multi-level approval
- Approval timeout
- Escalation

Notification Settings

- Email ON/OFF
- In-app notifications ON/OFF
- Reminder timing
- Approval notification
- Cancellation notification
- Maintenance notification

Room Settings

- Room types
- Facilities
- Buildings
- Floors
- Capacity rules
- Maintenance settings

User Settings

- Default role
- Account activation rules
- Password policy
- User deactivation rules

Security

- Session timeout
 - Password policy
 - Login attempt limit
 - JWT configuration
 - Security logs
-

34. ACCESS CONTROL SETTINGS

Create:

Settings → Roles & Permissions

Super Admin can manage:

Role

Select role.

Module

Select module.

Actions

Turn ON/OFF:

- View
- Create
- Edit
- Delete
- Approve
- Reject
- Cancel
- Export
- Manage
- Configure

Scope

Select:

- All
- Own
- Department
- Building
- Floor
- Room
- Custom

Example:

Room Manager

Room Module:

View = ON

Create = OFF

Edit = ON

Delete = OFF

Maintenance = ON

Export = OFF

Scope:

Building = Head Office

Floor = 2nd Floor

Rooms = Selected Rooms

35. PERMISSION INHERITANCE

Support:

Role permissions + User-specific overrides

Example:

Role:

Room Manager

User-specific override:

Export Rooms = ON

The final effective permission should be calculated by the backend.

Create an:

Effective Permissions

screen showing exactly what the user can and cannot access.

36. USER ACCESS PREVIEW

Admin should be able to select a user and see:

User Access Summary

Dashboard

- View ✓

Meeting Rooms

- View ✓
- Create ✕
- Edit ✓
- Delete ✕
- Maintenance ✓

Bookings

- View ✓
- Create ✓
- Edit ✓
- Cancel ✓

Approvals

- View ✕
- Approve ✕

Reports

- View ✓
- Export ✕

Also display:

Data Scope

Buildings:

- Head Office

Floors:

- 2nd Floor

Rooms:

- Board Room
- Conference Room A

This makes access management transparent.

37. SIDEBAR VISIBILITY

Sidebar must be dynamically generated from effective permissions.

If user does not have:

`Reports.View`

do not show Reports menu.

If user has:

`MeetingRooms.View`

but not:

`MeetingRooms.Create`

show Meeting Rooms but hide Add Room button.

If user has:

`MeetingRooms.Edit`

show Edit.

If user has:

MeetingRooms.Delete

show Delete.

However:

Frontend hiding is only for UX. Backend authorization is mandatory.

38. DASHBOARD

KPI Cards:

- Total Meeting Rooms
- Available Rooms
- Occupied Rooms
- Today's Meetings
- Upcoming Meetings
- Pending Approvals
- Approved Meetings
- Cancelled Meetings
- Room Utilization %
- Most Used Room

Charts:

- Daily Meeting Trend
- Weekly Meeting Trend
- Monthly Meeting Trend
- Room Utilization
- Department-wise Meetings
- Room-wise Booking Count
- Peak Meeting Hours
- Meeting Status Distribution

All analytics must be calculated from actual booking data.

No dummy analytics.

39. REPORTS

Reports:

- Daily Meeting Report

- Weekly Meeting Report
- Monthly Meeting Report
- Quarterly Meeting Report
- Yearly Meeting Report
- Room Utilization
- Department-wise Bookings
- Employee-wise Bookings
- Peak Hours
- Cancelled Meetings
- Approval Performance

Filters:

- Date range
- Department
- Building
- Floor
- Room
- Organizer
- Status
- Approval status

Export:

- Excel
- CSV
- PDF

40. NIRMAAN FISCAL YEAR

Use:

Q1

April – June

Q2

July – September

Q3

October – December

Q4

January – March

Financial year:

April – March

41. ACTIVITY / AUDIT LOG

Record:

- Login
- Logout
- User creation
- User update
- User activation
- User deactivation
- Role assignment
- Permission change
- Department creation
- Department update
- Room creation
- Room update
- Room deletion
- Room maintenance
- Booking creation
- Booking modification
- Booking cancellation
- Approval
- Rejection
- Notification
- Settings changes

Fields:

- User
- Action
- Module
- Record ID
- Old Value
- New Value
- Date/time
- IP
- User Agent / relevant system information

Audit logs must be immutable for normal admins.

42. DATABASE

Use normalized relational database.

Core tables:

```
roles
permissions
role_permissions
user_permissions
users
departments
meeting_rooms
room_facilities
facilities
room_managers
room_maintenance
bookings
booking_participants
booking_approvals
approval_workflows
approval_workflow_steps
notifications
email_templates
activity_logs
buildings
floors
system_settings
```

Avoid unnecessary duplicate tables.

Use foreign keys.

Use indexes for:

- booking date
- room ID
- start time
- end time
- status
- department
- user

- approval status
-

43. BOOKING DATABASE DESIGN

Booking must contain:

- ID
- Booking Reference
- Meeting Title
- Date
- Start Time
- End Time
- Room ID
- Organizer ID
- Department ID
- Participant Count
- Purpose
- Notes
- Booking Status
- Approval Status
- Created By
- Updated By
- Created At
- Updated At

Participants must be stored separately.

Approvals must be stored separately.

Audit history must be stored separately.

44. API SECURITY

Every API must implement:

- JWT authentication
- Role authorization
- Permission authorization
- Resource-level authorization
- Input validation
- SQL injection protection
- Rate limiting where appropriate
- Secure error handling

- CORS configuration
- Request logging

Never trust:

- User ID
- Department ID
- Room ID
- Role
- Permission

received from the frontend without backend validation.

45. REST API

Authentication:

```
POST /api/auth/login
POST /api/auth/logout
GET /api/auth/me
POST /api/auth/refresh
```

Dashboard:

```
GET /api/dashboard/summary
GET /api/dashboard/room-utilization
GET /api/dashboard/meeting-trend
```

Rooms:

```
GET /api/rooms
POST /api/rooms
GET /api/rooms/:id
PUT /api/rooms/:id
DELETE /api/rooms/:id
GET /api/rooms/available
POST /api/rooms/:id/maintenance
```

Bookings:

```
GET /api/bookings
POST /api/bookings
GET /api/bookings/:id
PUT /api/bookings/:id
POST /api/bookings/:id/cancel
POST /api/bookings/:id/reschedule
```

Approvals:

```
GET /api/approvals
POST /api/approvals/:id/approve
POST /api/approvals/:id/reject
```

Departments:

```
GET /api/departments
POST /api/departments
PUT /api/departments/:id
DELETE /api/departments/:id
```

Users:

```
GET /api/users
POST /api/users
GET /api/users/:id
PUT /api/users/:id
PATCH /api/users/:id/status
DELETE /api/users/:id
```

Roles:

```
GET /api/roles
POST /api/roles
PUT /api/roles/:id
DELETE /api/roles/:id
GET /api/roles/:id/permissions
PUT /api/roles/:id/permissions
```

Permissions:

```
GET /api/permissions
GET /api/users/:id/effective-permissions
PUT /api/users/:id/permissions
```

Notifications:

```
GET /api/notifications
PUT /api/notifications/:id/read
PUT /api/notifications/read-all
```

Calendar:

```
GET /api/calendar
```

Reports:

```
GET /api/reports/daily
GET /api/reports/weekly
GET /api/reports/monthly
GET /api/reports/quarterly
GET /api/reports/yearly
GET /api/reports/room-utilization
GET /api/reports/department
GET /api/reports/employee
GET /api/reports/peak-hours
```

Settings:

```
GET /api/settings
PUT /api/settings
GET /api/settings/booking-rules
PUT /api/settings/booking-rules
GET /api/settings/approval-workflow
PUT /api/settings/approval-workflow
GET /api/settings/email-templates
PUT /api/settings/email-templates
```

46. UI DESIGN

Use a professional Nirmaan enterprise UI.

IMPORTANT COLOR RULE

Use ONLY the existing approved Nirmaan portal color palette.

Do not introduce random colors.

Do not use:

- Purple
- Pink
- Neon gradients
- Random dashboard colors
- Unapproved branding colors

Charts, buttons, cards, status indicators and components should use appropriate shades/tints derived from the existing Nirmaan portal colors.

Do not change Nirmaan branding.

47. UI STRUCTURE

Desktop-first responsive design.

Layout:

```
Sidebar
+
Header
+
Page Content
```

Header:

- Portal Name
- Search where required
- Notifications
- User Profile
- Theme switch
- Logout

Sidebar:

Dashboard
Meeting Calendar
Book Meeting Room
My Bookings
Approval Requests
Meeting Rooms
Departments
Users
Roles & Permissions
Reports
Activity Logs
Settings

Show menu items based on effective permissions.

48. MEETING ROOM UI

Room listing should support:

- Grid view
- List view
- Search
- Filter
- Sort
- Building
- Floor
- Capacity
- Facilities
- Status

Room card:

- Room Image
- Room Name
- Room Code
- Capacity
- Building
- Floor
- Facilities
- Status
- Current availability
- Actions

49. BOOKING UI

Create a clean multi-step booking interface.

Step 1

Date & Time

Step 2

Participants & Requirements

Step 3

Available Rooms

Step 4

Meeting Details

Step 5

Review

Step 6

Submit

Show a clear summary before submission.

50. ERROR HANDLING

Use clear enterprise messages.

Double booking:

This room is already booked for the selected time. Please select another room or time.

Maintenance:

This room is currently under maintenance and cannot be booked.

Capacity:

The selected room does not have enough seating capacity.

No availability:

No meeting rooms are available for the selected time and requirements.

Permission:

You do not have permission to perform this action.

Inactive user:

Your account is inactive. Please contact the administrator.

51. RESPONSIVE DESIGN

Support:

- Desktop
- Laptop
- Tablet
- Mobile

However, optimize the primary experience for office desktop usage.

Tables should support horizontal scrolling where necessary.

Forms must remain usable on smaller screens.

52. LIGHT / DARK MODE

Support:

- Light Mode
- Dark Mode
- System Mode

But both themes must remain within the approved Nirmaan branding palette.

Do not introduce unrelated colors in dark mode.

53. EMPTY STATES

Every module must have useful empty states.

Examples:

No rooms:

No meeting rooms found.

No bookings:

You don't have any meeting bookings yet.

No approvals:

No approval requests available.

No notifications:

You're all caught up.

54. PERFORMANCE

Use:

- Pagination
- Server-side filtering
- Server-side sorting
- Lazy loading
- API caching where appropriate
- Database indexes
- Optimized Sequelize queries

Do not load thousands of bookings into the browser unnecessarily.

55. IMPORTANT BUSINESS RULES

1. Never allow overlapping room bookings.
2. Room capacity must support participant count.

3. Maintenance rooms cannot be booked.
 4. Inactive rooms cannot be booked.
 5. Cancelled bookings release the room.
 6. Booking status and approval status must remain separate.
 7. Approval actions must be audited.
 8. User permissions must be enforced on backend.
 9. Resource-level access must be enforced on backend.
 10. Dashboard data must come from real booking records.
 11. Reports must come from real booking records.
 12. No demo analytics.
 13. No unrelated modules.
 14. Historical booking data must remain auditable.
 15. Deactivated users must not lose historical records.
 16. Email must be triggered according to configured workflow.
 17. Confirmation email must be sent after final approval.
 18. Room managers can only manage assigned rooms/resources.
 19. Admin permissions must be configurable by Super Admin.
 20. Super Admin has complete system access.
 21. Normal Admin must never automatically receive unrestricted Super Admin permissions.
 22. Permission changes must create audit logs.
 23. Settings changes must create audit logs.
 24. Critical operations must require confirmation.
 25. Backend authorization must always be stronger than frontend visibility.
-

56. PROJECT STRUCTURE

```
Meetspace.Nirmaan/  
|  
├─ backend/  
|   └─ src/  
|       ├── config/  
|       ├── models/  
|       ├── controllers/  
|       ├── routes/  
|       ├── middleware/  
|       ├── services/  
|       ├── permissions/  
|       ├── validators/  
|       ├── notifications/  
|       ├── email/  
|       ├── reports/  
|       ├── utils/  
|       ├── migrations/  
|       ├── seeders/  
|       └─ app.js
```

```
|
|
|   └─ server.js
|
|   └─ .env
|   └─ package.json
|
└─ frontend/
    └─ src/
        └─ api/
        └─ components/
        └─ context/
        └─ hooks/
        └─ layouts/
        └─ permissions/
        └─ routes/
        └─ pages/
            └─ Dashboard/
            └─ Calendar/
            └─ BookRoom/
            └─ MyBookings/
            └─ Approvals/
            └─ Rooms/
            └─ Departments/
            └─ Users/
            └─ RolesPermissions/
            └─ Reports/
            └─ ActivityLogs/
            └─ Settings/
        └─ theme.js
        └─ App.jsx
        └─ main.jsx
    └─ package.json
```

57. DEVELOPMENT APPROACH

Build this as a real production application.

Do not create a static prototype.

Do not hard-code dashboard numbers.

Do not hard-code room availability.

Do not hard-code users.

Do not hard-code permissions.

Do not create fake approval states.

Do not create fake reports.

All major functionality must communicate with the backend REST API and MySQL database.

58. IMPLEMENTATION ORDER

Build in this order:

Phase 1

- Project setup
- Database
- Authentication
- JWT
- Roles
- Permissions

Phase 2

- User Management
- Department Management
- Buildings
- Floors
- Facilities

Phase 3

- Meeting Room Master
- Room Manager assignment
- Maintenance

Phase 4

- Booking engine
- Availability engine
- Double-booking prevention

Phase 5

- Approval workflow
- Notifications
- Email workflow

Phase 6

- Calendar
- My Bookings
- Approval Requests

Phase 7

- Dashboard
- Reports
- Exports

Phase 8

- Activity Logs
- Admin Settings
- Permission Management
- Resource-level access

Phase 9

- Security testing
- Permission testing
- Booking conflict testing
- Responsive testing
- Production optimization

59. ACCEPTANCE CRITERIA

Before considering the project complete, verify:

Authentication

- Login works
- Logout works
- JWT works
- Inactive users cannot login

Permissions

- Role permissions work
- User-specific permissions work
- Resource-level permissions work
- Backend blocks unauthorized actions
- Sidebar dynamically changes
- Buttons dynamically change

Rooms

- Room creation works
- Room editing works
- Room activation/deactivation works
- Maintenance works
- Facilities work
- Room manager scope works

Bookings

- Availability works
- Capacity validation works
- Double booking is impossible
- Maintenance blocks booking
- Cancellation releases room

Approval

- Department approval works
- Admin approval works
- Configurable workflow works
- Approve/reject works
- Approval history works

Email

- Request email sent
- Approval email sent
- Rejection email sent
- Confirmation email sent after final approval
- Cancellation email sent

Reports

- Reports use real database data
- Filters work
- Pagination works
- Excel export works
- CSV export works
- PDF export works

Audit

- Every important action is logged
- Permission changes are logged
- Settings changes are logged
- Approval actions are logged

60. FINAL PRODUCT REQUIREMENT

The final application must feel like a **real internal enterprise product of Nirmaan Organization**, not a generic room-booking demo.

The most important design principle is:

Simple for Employees, Powerful for Admins, Fully Controlled by Super Admin.

Employees should see only the functionality they need.

Department Heads should see department-related functionality.

Room Managers should see only their assigned rooms.

Admins should receive only the administrative permissions assigned to them.

Super Admin should have complete control over:

- Users
- Departments
- Rooms
- Buildings
- Floors
- Facilities
- Bookings
- Approvals
- Roles
- Permissions
- Resource Access
- Notifications
- Email Templates
- Reports
- Audit Logs
- System Settings

Every access decision must be controlled through the centralized permission engine.

The application must maintain a **single source of truth** for rooms, availability, bookings, approvals, notifications, reports and audit logs.

The architecture must remain modular so that **Meetspace.Nirmaan can later be extracted as a standalone Meeting Room Management module and integrated into another Nirmaan portal without major architectural changes.**